

Lectures 19,20

MINIMUM WEIGHT SPANNING TREE ALGORITHMS

1. Meaning of the spanning tree of a connected, undirected graph. Note that such a graph does not, in general, have a unique spanning tree.

2. Weighted graph: $w: E \rightarrow \mathbb{R}$. That is, a weight is associated with every edge.

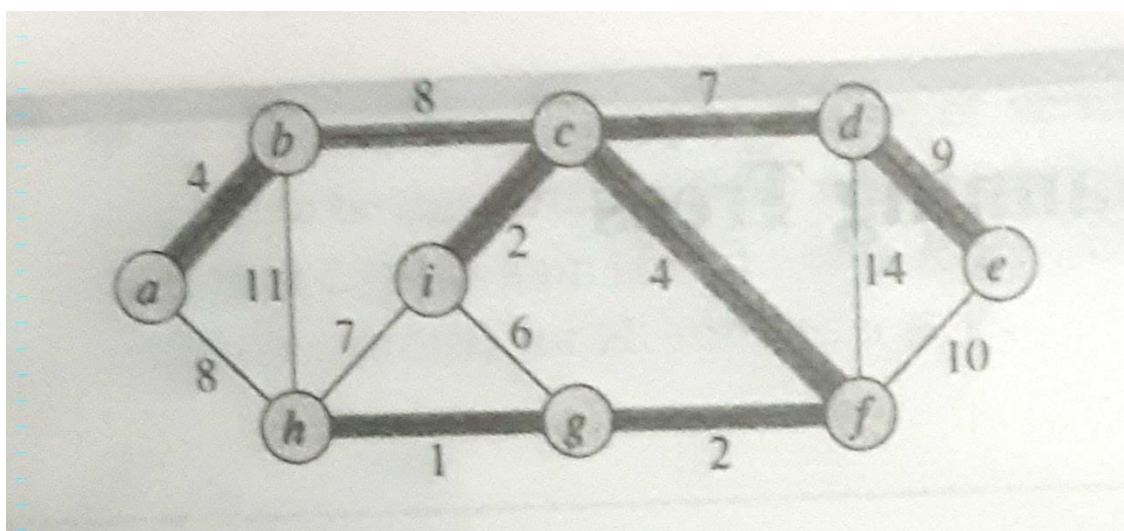
Example: Suppose that, in an electronic circuit, you need to connect N points, in such a way that the total 'cost' of these connections is minimized. This problem can be modelled using weighted graph, and the solution would then be a minimum weight spanning tree of a graph.

→ We represent all possible paths between these N points as a connected, weighted graph. Each such possible path is an edge in the graph, and the weight of the edge represents the 'cost' of the connection.

→ The connection pattern will define a spanning tree, say T . [Why?] So the total cost of these connections is given by: $w(T) = \sum_{(u,v) \in T} w(u,v)$.

→ What we need to compute, therefore, is a minimum weight spanning tree of the weighted graph which represents the N circuit points.

Example:



[BTW/GK: When do we say that a graph is '**planar**'?]

A generic MST algorithm:

```
GENERIC-MST( $G, w$ )  
  // Note  $A$  is a set of edges  
   $A = \text{NULL};$   
  while  $A$  does not form a spanning tree  
    find edge  $(u, v)$  which is safe for  $A$ ; // Main step  
     $A = A \cup \{(u, v)\};$ 
```

Loop invariant: Prior to each iteration, A is a subset of some minimum (weight) spanning tree (MST) of G .

Definitions:

Edge (u, v) is **safe** for A if, upon adding that edge to A , the loop invariant continues to be satisfied. That is, $A \cup \{(u, v)\}$ is also a subset of some MST of G .

A cut $(S, V - S)$ of an undirected graph $G = (V, E)$ is a partition of V . An edge (u, v) crosses the cut if it has one end-point in S and the other in $V - S$.

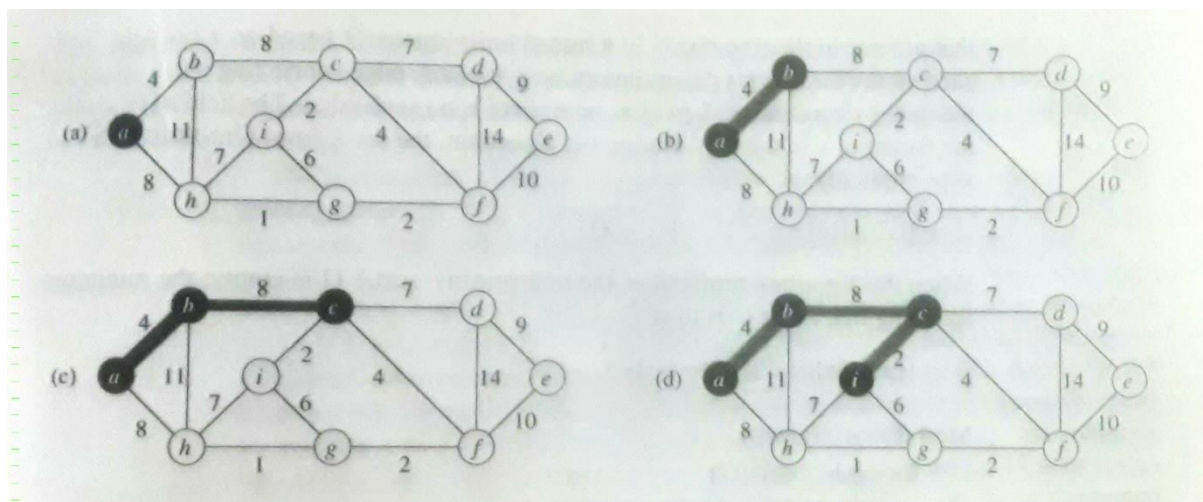
A cut respects a set of edges A if no edge in A crosses the cut.

An edge is a light edge crossing a cut if its weight is $\leq$ weight of any other edge crossing the cut.

Main property which is useful in the **main step**:
[Theorem 23.1 in CLRS]

Let G, w & A be as in the algorithm **GENERIC-MST**. Let $(S, V - S)$ be any cut of A that respects A . **Then any light edge crossing that cut is a safe edge for A .**

The following figure makes this concept clear.



Characteristics of Prim's algorithm:

1. Edges in A form a single tree, which grows from the root, one edge at a time.
2. $A = \{ (u,v) \mid u = \text{pred}[v], v \in V - \{r\} - Q \}$ → Note Q is indirectly giving A.
3. $(V - Q, Q)$ forms a cut, every time the **while** loop is entered. Q is reduced by one vertex every time.
4. Why is this a 'greedy algorithm'? → In every iteration, we add the minimum weight edge crossing the cut (**light edge**).
5. Q is to be implemented as a (minimum) priority queue based on $\text{key}[v]$. Binary MIN-HEAP can be used to implement Q.
6. For all vertices $v \in Q$, if $\text{pred}[v] \neq \text{NIL}$, then $\text{key}[v]$ is the weight of the light edge $(v, \text{pred}[v])$ connecting v to a vertex in A, the tree built so far.

Running time: With Q implemented as in 5 above, the algorithm is $O(E \lg V)$.

[BTW/GK: The other standard MST algorithm, which follows the same generic form, is known as '**Kruskal's algorithm**'.]